

Linux Firewall Configuration & Port Scan Defense

iptables Rule Management, Port Allow/Deny Policies, and Nmap Scan Mitigation

`iptables` is the standard command-line firewall utility in Linux that inspects network packets against a set of rules structured into chains (`INPUT` , `OUTPUT` , `FORWARD`).

Below are rules for managing ports and a demonstration showing how `iptables` blocks port scans (e.g., Nmap).

1. Simple iptables Rules (Allow / Deny Specific Ports)

Execute these commands with root privileges (`sudo`) on your server machine.

Core Allow & Block Rules

Objective	Command	Explanation
Allow HTTP (Port 80)	<code>sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT</code>	Appends (<code>-A</code>) a rule to allow incoming TCP traffic on port 80.
Allow SSH (Port 22)	<code>sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT</code>	Keeps SSH accessible before blocking other ports.
Allow Established Traffic	<code>sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT</code>	Ensures ongoing responses and established connections are not dropped.
Deny FTP (Port 21)	<code>sudo iptables -A INPUT -p tcp --dport 21 -j DROP</code>	Silently drops any connection requests targeting port 21.
Deny MySQL (Port 3306)	<code>sudo iptables -A INPUT -p tcp --dport 3306 -j REJECT</code>	Explicitly rejects MySQL connection requests with an ICMP response.

Setting a Default DROP Policy

To secure the firewall, it is best practice to allow specific essential ports (e.g., 22, 80, 443) and set the default policy to drop everything else:

```
# Set default policy for inbound traffic to DROP
sudo iptables -P INPUT DROP
```

2. Demonstrating Blocking a Port Scan Attempt

This demonstration compares port scan results from an attacker machine (e.g., Kali Linux running Nmap) against a target server **before** and **after** configuring `iptables` .

Phase 1: Port Scan BEFORE iptables Defense

On the **Attacker Machine**, scan the target IP (`192.168.1.50` or your lab target):

```
nmap -sS -p 21,22,80,3306 192.168.1.50
```

Output:

```
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
3306/tcp  open  mysql
```

Result: The scanner determines all target ports are open and active.

Phase 2: Configure iptables on Target Server

On the **Target Machine**, apply rules to explicitly allow SSH and HTTP, but drop or reject other ports:

1. Allow Loopback and Established Connections:

```
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
```

2. Allow Authorized Services (Ports 22 & 80):

```
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

3. Block Probed Ports (Ports 21 & 3306):

```
sudo iptables -A INPUT -p tcp --dport 21 -j DROP
sudo iptables -A INPUT -p tcp --dport 3306 -j REJECT
```

4. Enforce Default Drop Policy:

```
sudo iptables -P INPUT DROP
```

Phase 3: Port Scan AFTER iptables Defense

Run the exact same Nmap scan from the **Attacker Machine**:

```
nmap -sS -p 21,22,80,3306 192.168.1.50
```

Post-Defense Scan Results:

PORT	STATE	SERVICE	
21/tcp	filtered	ftp	<-- "DROP" rule completely hides the port (no reply)
22/tcp	open	ssh	<-- "ACCEPT" rule allows legitimate traffic
80/tcp	open	http	<-- "ACCEPT" rule allows legitimate traffic
3306/tcp	closed	mysql	<-- "REJECT" rule sends ICMP/TCP RST response

Key Differences Between DROP and REJECT

- **DROP (Port 21):** The target ignores incoming `SYN` packets without responding. Nmap reports the state as `filtered` because it times out waiting for a reply. This slows down automated scanning scripts.
- **REJECT (Port 3306):** The target actively sends back a `RST` (Reset) or ICMP error packet. Nmap immediately flags the port as `closed`.